TOMWare.M: Selective Pintool-Based Countermeasures against Anti-Instrumentation Techniques in Windows Executables

Resumo

Dynamic Binary Instrumentation (DBI) enables observation of program behavior at runtime but introduces artifacts that may reveal the analysis environment. In this context, this article presents TOMWare.M, an open-source *Pintool* based on Intel Pin for Windows, with five independently selectable countermeasures for debugging indicators, process and module enumeration, environment-variable names, memory-signature comparisons, and timing. The evaluation combined five controlled detectors and 3,100 executions of 31 Portable Executable (PE) binaries, including benign applications and malicious samples. The timing evaluation retained all 90 attempts across three conditions. Under the same artificial workload, scores remained at or below the calibrated threshold in 30/30 compensated attempts, compared with 11/30 without compensation. However, supplementary tests identified TickCount regressions in 4/10 compensated runs and alertable waits with no observed return before the test cutoff. A benign demonstrator developed by the authors restored a controlled file operation after process filtering. The results support indicator suppression through the evaluated interfaces without establishing general temporal consistency or equivalent work. Behavioral recovery in independently obtained evasive software remains an open validation step.

Palavras-chave: Instrumentação binária dinâmica, Anti-instrumentação, Intel Pin, Análise de malware, Proteção de software, Executáveis Windows

1 Introduction

Evasive malware and protected executables may alter their behavior according to the environment in which they execute. Operations such as payload decryption, dynamic API resolution, code reconstruction, and the activation of environment-dependent paths may remain concealed during static inspection and become observable only at runtime. In this context, Dynamic Binary Instrumentation (DBI) enables analysis routines to be inserted into compiled programs without requiring source code, recompilation, or permanent modification of the executable. Intel Pin, for example,

provides fine-grained access to instructions, registers, control transfers, function calls, and memory operations [1].

However, the visibility provided by DBI is accompanied by observable instrumentation artifacts. Dynamic translation and auxiliary analysis routines may introduce memory regions, libraries, processes, internal data structures, environment variables, and additional interactions with the operating system. Instrumentation may also alter execution intervals and resource consumption. Evasive programs can inspect these differences through anti-instrumentation checks to determine whether they are being analyzed. Upon detecting an indicator, a program may terminate, delay execution, follow an alternative path, or suppress relevant behavior. These responses reduce the completeness and fidelity of the evidence collected through dynamic analysis [2, 3].

The relevance of this problem has been demonstrated at different experimental scales. Polino et al. identified at least one anti-instrumentation technique in 15.6% of 7,006 analyzed samples [4]. Subsequently, Kim et al. examined 763,985 Windows malware samples collected between 2017 and 2020 and reported that an average of 16.21% employed anti-analysis techniques [5]. Although the latter study covers mechanisms beyond DBI detection, its findings reinforce that execution-environment awareness remains a recurring challenge for automated malware analysis.

Previous studies have documented different observation surfaces through which DBI environments can be identified. Santos Filho et al. systematized 26 evasion techniques and found that only a subset had corresponding countermeasures documented in the literature [3]. Park and Park described 24 Windows-oriented detection techniques involving timing overhead, *code-cache* artifacts, memory signatures, Just-in-Time (JIT) compiler characteristics, and environmental indicators [6]. Lee et al. investigated anti-VM and anti-DBI mechanisms employed by commercial software protectors and showed that DBI-based analysis and unpacking require countermeasures against the checks performed by protected code [7].

Despite these advances, isolating the effects of individual countermeasures remains a methodological challenge. Existing approaches commonly emphasize detection coverage or the bypass of checks associated with particular software protectors [4, 7, 8]. When multiple protections are enabled simultaneously, an observed functional or operational change cannot be readily attributed to a specific mechanism. This limits the independent assessment of functional effectiveness, compatibility, and operational cost. A countermeasure-oriented design, in which each protection can be activated and evaluated separately, establishes a more direct relationship among the detection surface, the implemented response, and the effects resulting from its activation.

Against this background, this article presents **TOMWare.M**, an open-source *Pintool* developed on Intel Pin for Windows.[1] The tool provides five independently selectable countermeasures. `AntiDebug` normalizes selected debugging indicators maintained in the Process Environment Block (PEB). `ProcessEnum` filters configured process and module names through the monitored enumeration and query interfaces. `SanitizePinEnvVars` renames selected entries in the environment block. `InstMemcmpMask` modifies the results of comparisons against configured signatures

[1] Source repository: https://github.com/TOMWare-analises/TOMWare. Build provenance and data identifiers are described in Section 7.4.

without altering the compared contents. In turn, `SkewMask` adjusts selected timing sources using the excess duration measured around intercepted waits.

Because these surfaces expose different artifacts, the countermeasures employ different intervention mechanisms. TOMWare.M combines process-structure normalization, enumeration filtering, environment-block sanitization, comparison-routine wrapping, and timing adjustment. Each module operates within an explicitly defined scope, allowing its functional and operational effects to be examined separately.

Based on this design, the study addresses the following research questions:

- **RQ1:** To what extent does each TOMWare.M countermeasure suppress its targeted Intel Pin indicator through the monitored interface while preserving the execution of the corresponding controlled detector?
- **RQ2:** What operational compatibility outcomes and changes in execution time and resource utilization are observed when each countermeasure is individually applied to benign and malicious Windows executables?

The evaluation distinguishes controlled functional validation from operational measurements. Five detector programs served as functional oracles, each exercising the surface associated with one countermeasure. The operational campaign comprised 31 Windows Portable Executable (PE) binaries, including 17 benign applications and 14 malicious samples. For each of the 155 binary-countermeasure combinations, ten executions were performed with the module disabled and ten with it enabled, in alternating order, totaling 3,100 executions. Persistent programs were observed during a fixed 10-s window.

Supplementary tests addressed timing under matched workloads, concurrent waits, module-name filtering, ordinary comparison calls, and combined activation. These tests are presented separately from the operational campaign. A benign demonstrator developed by the authors and five native/instrumented execution pairs for benign programs provided additional behavioral observations. This evidence is restricted to the exercised conditions and does not establish equivalent work or behavioral recovery in independently obtained evasive software.

The contributions of this study consist of the implementation of five selectable mechanisms, the association between observation surfaces and countermeasures, and an evaluation that distinguishes functional evidence from operational measurements. This separation allows indicator suppression, preservation of an execution profile, and behavioral recovery to be interpreted as distinct outcomes.

The remainder of this article is organized as follows. Sections 2 and 3 present the background and anti-instrumentation surfaces. Sections 4 and 5 describe the tool design and implemented mechanisms. Section 6 presents the experimental protocol and results, followed by the discussion in Section 7 and the conclusions and future work in Section 8.

2 DBI Artifacts and Countermeasure Landscape

2.1 DBI Execution and Observable Artifacts

Dynamic Binary Instrumentation enables compiled programs to be observed and modified during execution without requiring source code, recompilation, or permanent modification of the executable. This technique supports activities such as reverse engineering, behavioral malware analysis, instruction tracing, memory inspection, and code unpacking [1, 9].

Frameworks such as Intel Pin and DynamoRIO use dynamic translation to mediate the execution of the analyzed program. In Intel Pin, application code is translated and instrumented by a JIT engine before being stored in a *code cache*. Instrumentation routines determine where analysis logic is inserted, whereas analysis routines execute when the corresponding events occur [1, 9]. This model provides fine-grained access to instructions, registers, control transfers, function calls, and memory operations. However, dynamic translation, cache allocation, and auxiliary components introduce observable properties that differ from native execution and depend on the framework and its version [6, 10].

Transparency concerns the preservation of properties observable by the analyzed program. Bruening et al. identified code, data, and concurrency as relevant dimensions of transparency and described timing as particularly challenging [11]. Transparency must nevertheless be distinguished from compatibility: a program may complete its execution while still identifying the instrumentation environment [9, 10].

Based on this distinction, TOMWare.M considers three evaluation dimensions. Functional effectiveness determines whether the targeted indicator is suppressed through the monitored surface. Operational compatibility determines whether countermeasure activation preserves the expected execution profile. Operational effects characterize the observed changes in execution time and resource utilization. Preserving an execution profile does not, by itself, establish that equivalent work was performed. This separation allows each countermeasure to be evaluated according to its intended behavior without interpreting execution completion as evidence of complete transparency.

2.2 Anti-Instrumentation Surfaces

Anti-instrumentation techniques inspect artifacts and behavioral differences introduced by DBI execution. D'Elia et al. organized these observations into categories involving timing, code pointers, memory contents and permissions, internal structures, operating-system interactions, exception handling, and translation defects [9]. Santos Filho et al. distinguished direct techniques, which search for explicit instrumentation artifacts, from indirect techniques, which exploit functional or resource-related differences [3]. Park and Park complemented this landscape by cataloging 24 Windows-oriented techniques [6].

Based on these classifications, TOMWare.M addresses five observation surfaces: debugging indicators, process and module enumeration, Pin-related environment variables, in-memory signature comparisons, and timing sources. Associating each

surface with a selectable countermeasure enables targeted intervention and separate examination of its functional and operational effects.

The activation point is also relevant because some checks may occur before the conventional program entry point. Padhy et al. described an evasion sequence involving Thread Local Storage (TLS) callbacks, process inspection, and environment-dependent checks [12]. Because these callbacks may precede the application's main logic, countermeasures targeting early process state or environment data must act before the corresponding indicators are inspected.

2.3 Detection and Countermeasure Approaches

Studies of DBI observability have identified different ways to recognize instrumentation. Falcón and Riva introduced eXait, a test suite that examines Intel Pin through memory signatures, component names, process information, code-cache properties, and timing measurements [13]. Kirsch et al. showed that programs can exploit internal Pin characteristics, including differences in execution state and code-cache behavior [14]. Palmaro and Franchina demonstrated that low-level address-space scanning can reveal memory-resident components associated with DBI [15].

Comparative studies have also shown that artifact exposure varies across frameworks and versions. D'Elia et al. evaluated different DBI systems using existing and newly introduced detection checks [10]. Park and Park investigated techniques targeting Windows instrumentation environments [6]. These studies characterize the detection landscape from which the surfaces addressed by TOMWare.M were selected.

In countermeasure development, Polino et al. introduced Arancino, a defensive layer implemented on Intel Pin [4]. COBAI proposes a broader architecture for addressing instrumentation and operating-system artifacts [16], whereas Peekaboo uses DBI to recover behavioral evidence from evasive Windows malware [17]. Other approaches focus on debugging indicators and checks employed by commercial protectors, including the work of Park et al. and Lee et al. [7, 8]. Apate and hardware-assisted debugging approaches also seek to conceal debugger-related evidence from analyzed programs [18, 19].

Evaluating these techniques involves both their occurrence and their effects on execution. Kim et al. investigated the presence and influence of anti-analysis checks on the execution flow of Windows malware [5]. Norby et al. measured the operational cost of anti-debugging techniques on Windows and Linux [20]. These studies motivate examining both the functional outcome of a countermeasure and the operational changes associated with its activation.

Architectural alternatives seek to reduce in-process artifacts by moving part of the observation mechanism outside the analyzed program. PEMU provides Pin-compatible instrumentation outside the virtual machine [21], whereas LibIHT employs hardware facilities to trace control-flow branches [22]. TOMWare.M retains the conventional Intel Pin execution environment and introduces selectable countermeasures at the *Pintool* level.

Within this landscape, TOMWare.M focuses on the explicit association between five detection surfaces and independently selectable mitigation mechanisms. Its evaluation separates controlled functional validation from compatibility measurements and

Tabela 1 Targeted anti-instrumentation surfaces and representative inspection mechanisms.

Surface	Observable indicator	Representative inspection mechanisms	References
Debugger state	PEB fields and responses to debugger-related queries.	`IsDebuggerPresent`, `CheckRemoteDebuggerPresent`, `NtQueryInformationProcess`, and direct PEB inspection.	[6–8, 20]
Processes and modules	Names, handles, and paths of DBI components.	`CreateToolhelp32Snapshot`, `Process32First/Process32Next`, `Module32First/Module32Next`, `GetModuleHandle`, and `NtQuerySystemInformation`.	[4, 6, 13]
Environment variables	Names or values introduced by the instrumentation loader.	Environment-query APIs, C Runtime Library (CRT) functions, and direct inspection of the environment block referenced by the PEB.	[3, 4, 6]
Memory signatures	DBI-related names, strings, permissions, or memory regions.	Direct scanning, `VirtualQuery`, `VirtualQueryEx`, `memcmp`, `wmemcmp`, and equivalent comparison routines.	[9, 10, 15]
Timing	Elapsed-time deviations associated with instrumented execution.	`QueryPerformanceCounter`, `GetTickCount`, suspension functions, `RDTSC/RDTSCP`, and comparisons across execution intervals.	[6, 11, 23]

operational effects. The study does not experimentally compare coverage, compatibility, or performance against the cited systems; this organization therefore characterizes the scope of the tool and its evaluation without establishing superiority over other approaches.

3 Targeted Anti-Instrumentation Surfaces

Dynamic Binary Instrumentation may expose artifacts related to process state, runtime components, environment data, memory contents, and timing. Programs may inspect these surfaces individually or combine different observations to decide whether to continue execution. TOMWare.M organizes its countermeasures around these five surfaces, associating each with a specific intervention mechanism.

Table 1 presents representative inspection mechanisms. These mechanisms characterize the detection landscape and do not imply that every listed interface is intercepted by the tool. Implementation coverage and the interfaces actually exercised in the experiments are specified in Section 5.

3.1 Debugger-State Indicators

Debugger-oriented checks inspect execution state through Windows APIs, native process-information queries, or direct access to the Process Environment Block (PEB). These inspections may expose related information without necessarily constituting independent checks. For example, the response of `IsDebuggerPresent` is derived from the `PEB.BeingDebugged` field. Other queries expose distinct properties and require specific interpretation.

Although Intel Pin is not a conventional debugger, protected programs may combine debugger-oriented checks with DBI-specific inspections [7, 8]. TOMWare.M

associates this surface with `AntiDebug`, whose intervention targets selected PEB fields and the API response derived from them. Indicators obtained independently of these fields remain outside the module's direct coverage.

3.2 Process and Module Artifacts

Instrumentation launchers and runtime components may remain observable through process lists, loaded-module enumerations, or queries for specific modules. Names such as `pin.exe`, `pinvm.dll`, and `pinipc.dll` may serve as inspection targets, depending on the Pin version and launch configuration [4, 6, 13].

Process enumeration and module queries provide distinct views of the environment. Filtering one interface does not eliminate artifacts exposed through other interfaces. TOMWare.M associates this surface with `ProcessEnum`, which filters configured names through selected Tool Help and module-query interfaces.

3.3 Environment Artifacts

Instrumentation initialization may introduce environment entries that remain accessible to the application. Their names and values depend on the framework, version, and execution configuration. Inspection may occur through Windows query APIs, CRT representations, or direct access to the environment block referenced by the PEB [3, 4]. The intervention point is therefore relevant when different environment representations are created during initialization.

TOMWare.M associates this surface with `SanitizePinEnvVars`, which renames startup entries whose names begin with the three-character sequence `PIN`, without requiring an underscore after that sequence. The transformation changes the original lookup names while preserving the entries and their values.

3.4 Memory-Resident Signatures

Runtime libraries, internal structures, and translated code may introduce identifiable contents into the instrumented address space. Programs may inspect memory regions and search for known strings through library routines or custom scanning loops [9, 10, 15].

TOMWare.M associates this surface with `InstMemcmpMask`, which changes the result of eligible comparisons when the second operand matches a configured signature and the comparison length matches that signature's expected length. The underlying buffers remain unchanged. The countermeasure's effect therefore depends on the path used by the application: direct reads, unintercepted routines, and custom comparison sequences may still access the original contents.

3.5 Timing Deviations

Dynamic translation, code-cache generation, and the execution of callbacks and analysis routines introduce additional work that may affect measured execution intervals. A program may compare elapsed time against a threshold, contrast the first execution of a region with subsequent cached executions, or correlate different timing sources

[6, 11, 23]. Scheduling, virtualization, and concurrent system load also influence these observations, making thresholds dependent on the measurement context.

TOMWare.M associates this surface with `SkewMask`, which accumulates excess duration measured around intercepted relative suspensions and adjusts selected timing sources. Its evaluation distinguishes the timing-detector outcome from clock consistency and preservation of overlapping and alertable-wait behavior.

The mapping between these surfaces and their countermeasures underpins the architecture presented in Section 4 and enables separate examination of the implemented mechanisms.

4 TOMWare.M Design and Execution Model

TOMWare.M is an open-source, countermeasure-oriented *Pintool* developed on Intel Pin for Windows [1]. Its architecture allows countermeasures to be activated individually or combined according to the observation surfaces relevant to the analyzed executable. Selection uses command-line parameters and requires neither recompilation nor permanent modification of the program. Figure 1 presents the components and execution flow.

The central controller parses the parameters, initializes instrumentation mechanisms, registers callbacks and replacement routines, and starts the target program under Intel Pin. The parameters `-dd`, `-dp`, `-de`, `-dm`, and `-do` activate `AntiDebug`, `ProcessEnum`, `SanitizePinEnvVars`, `InstMemcmpMask`, and `SkewMask`, respectively. The `-da` parameter activates all five countermeasures in the same execution.

In the experiments, TMW0 denotes the configuration with Pin and TOMWare.M loaded but all countermeasures disabled (`-q`). This condition includes `TOMWare.dll` and therefore does not correspond to Pin-only execution. The EMPTY condition uses Pin with an empty Pintool, identified as `EmptyPinTool.dll`, and is reported in the supplementary tests in which it was used.

In the operational campaign, each countermeasure was activated individually to allow assessment of its functional and operational effects. Combined activation also underwent a smoke test on the five controlled detectors, with one recorded execution per detector (Table 10). This test is separate from the operational campaign and was performed without the artificial workload introduced by `-go`.

The countermeasures employ initialization and intervention mechanisms specific to each surface. `AntiDebug` and `SanitizePinEnvVars` operate on process structures and environment data during application initialization. Modules that intercept APIs or library routines register image-loading callbacks and install replacements when the corresponding libraries and symbols become available. After initialization, each enabled module operates within the scope defined by its structures, interfaces, signatures, or timing sources.

Experimental orchestration is maintained outside the *Pintool* core. External scripts launch the baseline and enabled conditions, alternate execution order, collect time, CPU, and memory measurements, and produce CSV and JSON records. This organization distinguishes the interventions performed by the tool from the control and collection procedures used in the experimental evaluation described in Section 6.

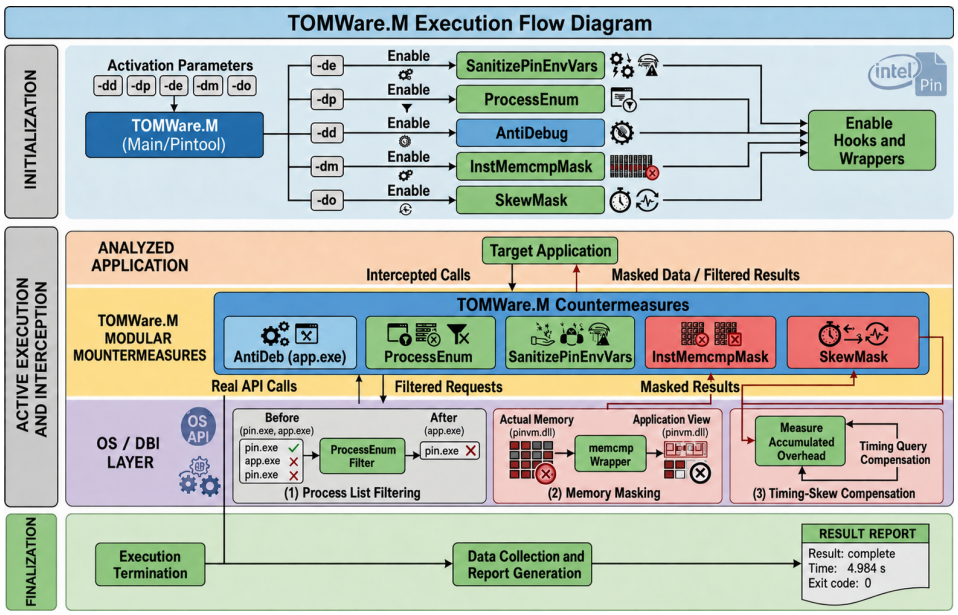


Figura 1 Countermeasure-oriented architecture and activation workflow of TOMWare.M.

5 Countermeasure Mechanisms

TOMWare.M employs different intervention mechanisms for the five observation surfaces addressed. Depending on the targeted indicator, a countermeasure normalizes process structures, filters information returned by enumeration interfaces, sanitizes environment-variable names, modifies memory-comparison results, or adjusts values provided by selected timing sources. This organization allows each intervention to be adapted to the characteristics of its observation surface.

Previous studies have shown that DBI detection encompasses debugger state, process and module information, environmental artifacts, memory signatures, and timing behavior [3, 6, 9, 10]. TOMWare.M associates each surface with an independently selectable countermeasure, allowing its functional behavior and operational effects to be examined separately.

The following subsections describe the activation point, monitored interfaces, transformation procedure, functional validation, and implementation boundaries of each module. Interfaces supported by the implementation are distinguished from those actually exercised by the tests, so that experimental evidence is associated with the evaluated paths. Suppressing an indicator through one interface does not imply that equivalent inspections performed through alternative mechanisms are also neutralized.

5.1 AntiDebug: PEB-Based Debugger-State Normalization

Anti-debugging checks may inspect execution state through Windows APIs, native process-information queries, and fields maintained in the *Process Environment Block* (PEB). Representative mechanisms include `IsDebuggerPresent`,

Tabela 2 Representative paired validation of `AntiDebug` under a simulated PEB stimulus.

Observed indicator	Reference `-gdb -q`	Enabled `-gdb -dd -q`	Effect
`IsDebuggerPresent()`	1	0	Derived
`PEB.BeingDebugged`	1	0	Direct
`PEB.NtGlobalFlag`	1	0	Direct
Reported oracle outcome	Complete	Complete	Preserved

Note: Values 1 and 0 represent positive and negative indicator observations, not raw PEB field values or counts of independent repetitions. Direct denotes a PEB field explicitly normalized by the countermeasure. Derived denotes the API response affected by normalization of `BeingDebugged`.

`CheckRemoteDebuggerPresent`, `NtQueryInformationProcess`, and direct inspection of the `BeingDebugged` and `NtGlobalFlag` fields [6, 8, 20]. Whereas broader debugger-hiding systems modify different properties of the debuggee or employ hardware-assisted isolation [18, 19], TOMWare.M implements a targeted user-space countermeasure for PEB-based indicators.

The `AntiDebug` module obtains the PEB address through the *Thread Environment Block* (TEB). It clears the `BeingDebugged` field and locates `NtGlobalFlag` using the architecture-specific offset. Before modifying the fields, it verifies that the destination memory is writable. The normalization procedure runs during application startup and again when the main executable image is loaded.

Because `IsDebuggerPresent` derives its result from `PEB.BeingDebugged`, normalizing this field also changes the API response without requiring direct function replacement. Thus, the modifications to `BeingDebugged` and `NtGlobalFlag` are direct, whereas the effect on `IsDebuggerPresent` results from the PEB modification.

The evaluated implementation does not directly replace `CheckRemoteDebuggerPresent` or `NtQueryInformationProcess`. Indicators obtained independently of the two normalized fields, including exception-based checks, debug-register inspection, and unsupported process-information classes, remain outside the module's direct coverage.

Functional validation used `TestAntiDebug.exe`, which combines an API-level query with direct inspection of the two PEB fields. Both conditions used the same detector, Intel Pin configuration, and simulated PEB stimulus, without an attached debugger. The reference condition used `-gdb -q`, whereas the countermeasure-enabled condition used `-gdb -dd -q`. Activating `AntiDebug` through `-dd` was therefore the only countermeasure setting changed between the paired executions.

Table 2 reports a representative paired observation and distinguishes direct field normalization from the derived API response.

In the representative pair, all three indicators changed from positive to negative after countermeasure activation. The detector reported completion in both conditions. These observations support suppression of the tested PEB-based indicators while preserving completion of the controlled detector.

The functional evidence is restricted to the indicators exercised under the simulated stimulus and does not establish behavior with an attached debugger. It also does

not establish suppression of indicators obtained through interfaces independent of the normalized structures. Complete oracle outputs and execution records are available in the reproducibility package.

5.2 ProcessEnum: Process and Module Artifact Filtering

Process and module enumeration can expose loaders, auxiliary libraries, and other components associated with an instrumentation environment [4, 6, 13]. On Windows, these inspections can be performed through the Tool Help API family, module-query functions, native system-information interfaces, and direct inspection of process structures.

The `ProcessEnum` module applies name-based filtering to enumeration results. For processes, it replaces `Process32First`, `Process32Next`, `Process32FirstW`, and `Process32NextW`. The base name of each entry is compared against the configured rule without distinguishing uppercase from lowercase characters. When an entry corresponding to `pin.exe` is encountered, the replacement advances the enumeration until it obtains an entry not included in the filter. Other entries are returned without modification.

The same procedure is applied to `Module32First`, `Module32Next`, `Module32FirstW`, and `Module32NextW`. The configured names are `pinvm.dll`, `pinipc.dll`, and `TOMWare.dll`. Matching entries are omitted from the sequence returned to the monitored program, whereas the remaining entries preserve their original enumeration order.

The module also replaces `GetModuleHandleA` and `GetModuleHandleW`. When the name argument is non-null, TOMWare.M extracts its base name and performs a case-insensitive comparison against the configured filter. Requests matching a monitored module receive a null handle. Other requests are forwarded to the original function, as are calls whose name argument is `NULL`.

The original functional oracle, `TestProcessEnum.exe`, enumerates processes through the Unicode interfaces and queries the monitored names through `GetModuleHandleA`. It does not invoke the `Module32` enumeration routines. In this experiment, activating `ProcessEnum` through `-dp` was the only difference between the evaluated configurations. The `pin.exe` count changed from eight occurrences in the baseline to zero with the countermeasure enabled (Table 3). The module names were absent in both conditions and therefore did not provide positive filtering controls.

To obtain a positive module-filtering control, the supplementary validation loaded a copy of `version.dll` in the temporary directory under the name `pinvm.dll`. This benign fixture exercises the name-filtering rule and is not an actual Pin component. It remained visible under native execution and TMW0 but was omitted from enumeration with `-dp` and `-da`. In all four conditions, `kernel32.dll` remained visible and enumeration terminated (Table 4).

A separate supplementary execution of the process oracle on the diagnostic guest reported `pin.exe` counts of 0, 3, 0, and 0 under native execution, TMW0, ProcessEnum, and combined activation, respectively. These records are distinct from the original eight-to-zero result and are not pooled as repetitions of that experiment.

Tabela 3 Original oracle-level functional validation of `ProcessEnum`.

Monitored artifact	**Baseline** without `-dp`	**ProcessEnum** with `-dp`	**Observed effect**
`pin.exe`	8	0	Filtered
`pinvm.dll`	0	0	Not observed
`pinipc.dll`	0	0	Not observed
`TOMWare.dll`	0	0	Not observed
Oracle verdict	Alert	OK	Target suppressed

Note: The counts in this table belong exclusively to the original oracle. An artifact absent from the baseline does not constitute a positive filtering control.

Tabela 4 Supplementary module-filtering and process-enumeration controls obtained in separate executions.

	Module control				**Separate oracle**
Condition	**Module count**	**Test module**	**kernel32**	**Ends**	`pin.exe`
Native	32	1	1	Yes	0
TMW0	31	1	1	Yes	3
ProcessEnum	30	0	1	Yes	0
Combined	30	0	1	Yes	0

Note: Test-module and kernel32 values indicate presence (1) or absence (0). The native condition does not use Pin. TMW0, ProcessEnum, and combined activation use, respectively, `-q`, `-dp -q`, and `-da -q`. The last column reports results from the separate process-oracle execution, not the module test. All supplementary executions reported exit code zero.

The supplementary controls support filtering of the tested module name, preservation of an entry not included in the filter, and termination of the exercised enumeration. These results do not independently validate every ANSI/Unicode First/Next path, every module-query variant, or the precise last-error convention. Complete oracle outputs and supplementary records are included in the reproducibility package.

The current `ProcessEnum` implementation does not intercept `NtQuerySystemInformation`, functions from the Process Status API (PSAPI), or direct inspection of internal process structures. Parent-child relationships, services, process identifiers, file paths, and artifacts not represented in the configured name list also remain outside its coverage.

5.3 SanitizePinEnvVars: Environment-Variable Sanitization

Environment variables may contain identifiers introduced during the initialization of an instrumentation environment. Previous studies have shown that variables inherited by an instrumented process can reveal the presence of a DBI loader or JIT engine

Tabela 5 Paired functional validation of the `SanitizePinEnvVars` countermeasure.

Oracle observation	**Baseline** without `-de`	**Sanitized** with `-de`	**Effect**
`PIN_CRT_TZDATA` (environment block)	Present	Absent	Sanitized
`PIN_CRT_TZDATA` (`_dupenv_s`)	Present	Absent	Sanitized
`PIN_APP_LD_LIBRARY_PATH`	Absent	Absent	Not observed
Oracle verdict	Alert	OK	Suppressed
Execution outcome	Complete	Complete	Preserved

Note: "Absent" in the sanitized configuration indicates that the original variable name was not resolved. The entry remained in the block under the modified prefix, with its value preserved. The two observations of the variable correspond to distinct inspection paths, not independent experimental repetitions.

[3, 4, 6, 10, 14]. Under Intel Pin, the process environment may contain entries such as `PIN_CRT_TZDATA`, `PIN_APP_LD_LIBRARY_PATH`, and `PIN_VM_LD_LIBRARY_PATH`. These entries can be retrieved through Windows APIs, C Runtime Library (CRT) functions, or direct inspection of the environment block referenced by the PEB.

The `SanitizePinEnvVars` module registers an application-start routine that accesses the process parameters referenced by the PEB, obtains the environment block, and traverses its null-terminated entries. When the first three characters of a name match `PIN` in a case-insensitive comparison, the routine verifies that the destination memory is writable and replaces those characters with `ZZZ`. The matching rule does not require an underscore after this sequence.

For the Pin variables considered in this study, the transformation changes the prefix from `PIN_` to `ZZZ_`. Because the replacement preserves the length, the entry boundaries and positions of subsequent entries remain unchanged. The procedure renames the variable without deleting its entry or modifying its value. CRT queries may reflect the change when their environment representation is initialized from the sanitized block; previously created copies are not updated by this operation.

Functional validation used `TestGetEnvironments.exe`, which searches for monitored variables through direct environment-block inspection and `_dupenv_s`. These procedures exercise one direct access path and one CRT-mediated path. Both conditions used the same detector and Pin configuration. Activating `SanitizePinEnvVars` through `-de` was the only difference between the paired executions.

Table 5 presents the results. The variable `PIN_CRT_TZDATA` served as a positive control because it was available in the baseline through both inspection paths.

In the baseline, `PIN_CRT_TZDATA` was identified through both paths, producing an alert. With the countermeasure enabled, neither path resolved the original name, and the oracle no longer issued the corresponding alert. Both executions completed. The entry `PIN_APP_LD_LIBRARY_PATH` was absent in both conditions and therefore does not constitute an additional positive sanitization control.

These results support suppression of the original name through direct environment-block inspection and `_dupenv_s`. They do not independently validate the interfaces `GetEnvironmentVariableA`, `GetEnvironmentVariableW`, or `getenv`. The oracle also

did not include explicit controls to demonstrate preservation of entries unrelated to the sanitization rule.

Coverage is restricted to entries matching the naming rule in the environment block processed during initialization. The transformation does not remove associated values, and previous copies, alternative identifiers, and other environmental artifacts remain outside its scope. Complete oracle outputs and execution records are included in the reproducibility package.

5.4 InstMemcmpMask: Memory-Signature Comparison Masking

DBI environments may introduce identifiable data structures, memory regions, and character strings into the address space of an instrumented process. Scanning techniques can use these artifacts to identify components associated with instrumentation [6, 10, 14, 15]. In the evaluated Intel Pin configuration, the monitored signatures were `PIN_`, `pin.exe`, `pinvm.dll`, and `pinipc.dll`. TOMWare.M also allows additional signatures to be supplied through a configuration file.

The `InstMemcmpMask` module examines the symbols exported by each loaded image and installs routine-specific wrappers when comparison routines are available and eligible for replacement. The configured list includes `memcmp`, `_memcmp`, `__acrt_memcmp`, `bcmp`, `memicmp`, `_memicmp`, `_memicmp_l`, `memcmp_s`, `wmemcmp`, `wmemcmp_s`, `RtlCompareMemory`, and `RtlEqualMemory`. This list describes the intended implementation coverage; the interfaces actually exercised are identified in the tests presented below.

Before invoking the original implementation, the wrapper examines the second operand and the comparison length, expressed in the units expected by the corresponding routine. When this operand exactly matches a monitored signature and the length matches the configured value, the wrapper returns an interface-specific non-match result. For conventional comparison routines, this result is nonzero. For `RtlCompareMemory`, it is a value smaller than the requested length, whereas `RtlEqualMemory` returns false. When the masking condition is not satisfied, the call is forwarded to the original routine.

Masking changes only the comparison result, leaving the memory operands unchanged. For functions that also express ordering, the forced nonzero result indicates inequality but does not necessarily reproduce the original lexicographic relationship between the operands.

Functional validation used `TestMemoryScan.exe`, which traverses accessible committed memory regions and compares candidate positions against the four monitored signatures through `memcmp`. The oracle executable, Pin configuration, signature file, and scanning procedure remained unchanged between conditions. The countermeasure-enabled configuration used `-dm`.

Table 6 presents the paired match counts. The reduction was calculated for each signature relative to its baseline count.

The baseline reported 255 matches, whereas the countermeasure-enabled condition reported none. Both oracle executions completed. This result supports suppression of

Tabela 6 Paired functional validation of the `InstMemcmpMask` countermeasure.

Signature	Baseline matches	Matches with `-dm`	Reduction
`PIN_`	212	0	100%
`pin.exe`	11	0	100%
`pinvm.dll`	14	0	100%
`pinipc.dll`	18	0	100%
Total	**255**	**0**	**100%**

Note: Counts represent signature matches reported through the `memcmp` calls issued by `TestMemoryScan.exe`. They are occurrences within the scan, not independent experimental repetitions.

Tabela 7 Focused comparison tests on corrected build `D55EC952`.

Interface	Target under native/TMW0	Target with `-dm`/`-da`	Ordinary controls
`memcmp`	Equal	Unequal	Preserved
`_memicmp`	Equal	Unequal	Preserved
`RtlCompareMemory`	Equal	Unequal	Preserved
`_memicmp_l`	Equal	Unequal	Preserved

Note: Ordinary controls used equal operands and eight-byte operands differing at the final byte. `RtlCompareMemory` returned eight and seven matching bytes, respectively. `memcmp` also passed a zero-length control. The `_memicmp_l` test used a null locale argument.

the configured signatures through the exercised `memcmp` path without modifying the scanned contents. It does not independently validate the other comparison interfaces.

Supplementary tests identified an argument-count defect in the wrapper present in historical build `4BBC6270`. The wrapper used three arguments to intercept `_memicmp_l`, whose interface includes a fourth locale argument, and the pass-through path to the original implementation failed. Corrected build `D55EC952`, associated with commit `5cc893f6`, uses four arguments and forwards the locale parameter.

On the corrected build, focused tests with `TestMemcmpVariants.exe` exercised four routines under native execution, TMW0 (`-q`), `-dm -q`, and `-da -q`. Each condition was executed once and exited with code zero. Target comparisons returned equality under native execution and TMW0, and inequality with masking enabled. Ordinary controls with equal and unequal operands preserved the expected results (Table 7).

The supplementary test did not locate the symbols for `RtlEqualMemory`, `wmemcmp`, `memcmp_s`, `wmemcmp_s`, or `bcmp` in the examined libraries; these paths remain unexercised. Other aliases, non-null locale arguments, and error-handling paths were not independently validated.

Corrected-build results are presented separately from the historical campaign. The original memory-scan counts, corpus measurements, and timing results remain associated with build `4BBC6270` and are not presented as measurements of the corrected DLL.

The countermeasure is limited to comparisons performed through replaceable routines, with the monitored signature supplied as the second operand and using its expected length. Direct scans, reversed operands, partial or transformed searches, inlined comparison loops, ordering-sensitive logic, and inspections of memory-page properties remain outside its coverage. Complete oracle outputs, signature files, and execution records are included in the reproducibility package.

5.5 SkewMask: Timing-Skew Compensation

Dynamic translation and the execution of analysis routines introduce delays that may alter the time perceived by an instrumented process [6, 9–11]. Evasive applications can measure execution intervals, compare initial JIT-translated executions with subsequent cached executions, or correlate different timing sources against an expected threshold [14, 23].

The `SkewMask` module replaces `Sleep`, `SleepEx`, and `NtDelayExecution`. For each intercepted relative delay, it records timestamps before and after the original operation using `QueryPerformanceCounter`. The observed interval is calculated using the frequency provided by `QueryPerformanceFrequency`, and the requested delay is converted to the same duration unit. The excess is calculated as

$$\delta_s = \max\left(0, \Delta t_{\mathrm{obs}} - \Delta t_{\mathrm{req}}\right), \tag{1}$$

where Δt_{obs} denotes the measured interval and Δt_{req} denotes the requested delay. The resulting excess is accumulated throughout execution. The maximum operation prevents a negative increment. This restriction alone does not guarantee monotonic adjusted clocks or accounting for overlapping delays without double counting.

The accumulated duration is subtracted from the values returned by `QueryPerformanceCounter`, `RtlQueryPerformanceCounter`, `GetTickCount`, `GetTickCount64`, `GetSystemTimePreciseAsFileTime`, and `QueryInterruptTimePrecise`. Before subtraction, the duration is converted to the unit required by the interface: performance-counter counts, milliseconds, or 100-nanosecond intervals. The consistency of these adjustments was examined separately in the wait tests presented below.

Functional validation used `TestOverhead.exe`. The oracle reads `GetSystemTimePreciseAsFileTime` before and after a 50-ms call to `Sleep` and multiplies the interval, in milliseconds, by 40. The resulting score is compared against the calibrated threshold

$$T = 1.5 \times (50 \times 40) = 3\,000. \tag{2}$$

A score above T produces a timing-anomaly verdict. A score at or below T indicates no targeted anomaly under the conditions of this oracle. The threshold does not represent a universal boundary for DBI detection.

Tabela 8 Matched-workload timing-oracle results: 30 attempts per configuration, with all 90 scores retained.

Configuration	Median score	Minimum maximum	Scores $\leq$ 3000
		2168.12	
TMW0	2433.285	2749.98	30/30
		2722.01	
GO	3083.210	3845.31	11/30
		2073.28	
GO+DO	2104.770	2146.11	30/30

Note: Wilson 95% confidence intervals for the proportion of scores at or below the threshold are: GO, [21.9%, 54.5%]; TMW0 and GO+DO, [88.6%, 100%]. Scores represent elapsed milliseconds multiplied by 40, not processor cycles.

The revised comparison retained the same artificial workload in the GO (`-go -q`) and GO+DO (`-go -do -q`) conditions, distinguishing them by SkewMask activation. The TMW0 condition (`-q`) provided the reference with TOMWare.M loaded and no artificial overhead. The detector, requested delay, scale factor, threshold, and Pin configuration remained unchanged. Thirty attempts were performed per condition, retaining all 90 scores. This design replaces the previous comparison between configurations with different workloads.

TMW0 includes Pin and TOMWare.M with all countermeasures disabled. A Pin-only or empty-Pintool condition was not collected for this same series of 30 attempts per configuration. The EMPTY condition appears in the complementary wait experiment, reported separately.

The contrast between GO and GO+DO supports suppression of the tested timing indicator while retaining the artificial workload. This result does not estimate total instrumentation overhead or establish temporal correctness for other workloads. All attempts produced scores, and none was flagged as timed out. However, the collector recorded exit code -1 throughout, leaving natural termination unverified. Individual scores and collection flags were retained in the experimental records.

The complementary experiment used ten attempts in each of four conditions: native execution, Pin with an empty Pintool (EMPTY), TMW0, and SkewMask (`-do -q`), without artificial overhead. Each run included two sequential 200-ms sleeps and two concurrent 200-ms sleeps in distinct threads released through a common barrier. Worker timestamps confirmed overlapping intervals. The oracle performed 256 QPC and TickCount reads during overlap and a subsequent 1,000-read burst. Non-alertable 200-ms and alertable 500-ms calls to `SleepEx` were also exercised, with an Asynchronous Procedure Call (APC) queued to interrupt the alertable wait.

Sequential and concurrent sleeps and non-alertable calls to `SleepEx` returned in all conditions. The subsequent 1,000-read bursts showed no regressions. During overlapping waits, however, TickCount regressed in four of the ten SkewMask runs. QPC did not regress, but it is also intercepted by the module and therefore does not provide an independent external time reference.

Tabela 9 Wait-model results: ten attempts per condition, without artificial overhead.

Condition	QPC regressions	TickCount regressions	Early APC return
Native	0/10	0/10	10/10
EMPTY	0/10	0/10	10/10
TMW0	0/10	0/10	10/10
SkewMask	0/10	4/10	0/10

Note: Regression columns count runs with backward readings during overlapping waits. Early APC return requires return value 192 and confirmed callback execution. All ten SkewMask runs reached the alertable-wait phase but were terminated at the 180-s cutoff without an observed return; the other conditions completed.

APC queueing succeeded under SkewMask without an observed alertable return or confirmed callback execution. The causes of the regressions and timeouts remain unconfirmed. These results do not establish correct accounting of overlapping delays, a globally coherent timeline, or preservation of alertable-wait semantics. Any correction to the compensation model requires renewed validation on the modified build.

The timing-score experiment directly exercised `Sleep` and `GetSystemTimePreciseAsFileTime`. The complementary tests also exercised `SleepEx`, `QueryPerformanceCounter`, and `GetTickCount`; the remaining intercepted interfaces were not independently validated. Direct `RDTSC` or `RDTSCP` reads, unsupported timing sources, external counters, absolute delays supplied to `NtDelayExecution`, and overhead accumulated outside intercepted relative-delay operations remain outside coverage.

5.6 Combined Activation

Simultaneous activation of all five countermeasures through `-da` was evaluated in a smoke test using the five controlled detectors. One execution was recorded per detector, with exit code zero in every case (Table 10).

The AntiDebug test retained the simulated stimulus enabled through `-gdb`. The timing test ran without the artificial workload introduced by `-go`; its result therefore constitutes an execution check with the modules combined, rather than additional evidence of timing suppression under a matched workload. The evidence from this test is restricted to the paths exercised by the detectors.

6 Experimental Evaluation and Results

The evaluation combines the historical operational campaign with supplementary functional and behavioral tests. Conditions, repetition counts, and observation windows are reported separately, preserving the correspondence between each experiment and its results.

Tabela 10 Combined-activation smoke test: one execution per detector, with exit code zero in all five executions.

Detector	Parameters	Observed outcome
AntiDebug	`-gdb -da -q`	Targeted indicators equal to zero
Process enumeration	`-da -q`	`pin.exe` absent
Environment	`-da -q`	Original targeted names unresolved
Memory scan	`-da -q`	Targeted match counts equal to zero
Timing	`-da -q`	Score 2040.7; no artificial workload

6.1 Experimental Environment

The host machine ran Windows 11 Home x64 with 16 GB of RAM and VMware Workstation Pro 25H2u1 (version 25.0.1). The analysis virtual machine ran Windows 10 x64 with 8 GB of RAM, four virtual CPUs, and a 60-GB virtual NVMe drive. The network adapter operated in *host-only* mode. A clean snapshot, identified as `tomware_go`, was restored between batches of malicious samples.

The historical campaign used Intel Pin 3.28.98749 for x64. The TOMWare.M DLL used in this campaign is identified by the prefix `4BBC6270` of its SHA-256 hash. The comparison-wrapper correction was evaluated separately on build `D55EC952`. Full identifiers and the limitations of reconstructing the historical build from source code are presented in Section 7.4.

The TMW0 condition consists of Pin with TOMWare.M loaded and all countermeasures disabled (`-q`). The main campaign compares this baseline with activation of one countermeasure at a time. This comparison characterizes the incremental effects of module activation without estimating total instrumentation overhead relative to native execution. Native execution and Pin with an empty Pintool were used in selected supplementary tests and are identified in their respective results.

6.2 Corpus and Evaluation Protocol

The historical campaign comprised 31 x64 Windows PE binaries: 17 benign applications and 14 malicious samples. The retained catalog records their sources as the Windows environment and MalwareBazaar, respectively. The acquisition history and selection rationale are not fully documented. The results characterize this corpus and do not estimate the prevalence of supported anti-instrumentation checks.

Historical reports describe ten short-lived and seven persistent benign binaries, together with four short-lived and ten persistent malicious binaries. These aggregate profile totals remain to be reconciled with the per-binary catalog metadata. They are reported separately from the valid counts contributing to individual metric summaries.

Each binary was evaluated separately with the five countermeasures. Every binary-countermeasure combination used ten process launches per condition, with the module disabled and enabled and the launch order alternated. The design comprised

$$31 \times 5 \times 2 \times 10 = 3{,}100$$

canonical process launches and 155 binary-countermeasure combinations. Supplementary tests are excluded from this total.

The campaign documentation additionally reports 260 diagnostic and re-execution records. These records are excluded from the canonical launch count and consolidated operational results.

The adopted operational criterion was recorded completion for short-lived binaries and continued activity throughout a fixed 10-s observation window for persistent binaries. Reaching the planned endpoint of a persistent observation was treated as an observation-window outcome. These criteria do not establish completion of useful tasks or equivalent application progress.

During the revision audit, wall-clock comparisons were considered applicable to naturally completed short-lived pairs with valid timestamps. Persistent CPU and memory comparisons required paired resource collection. These applicability rules were formalized during revision rather than preregistered.

For each binary and applicable metric, condition medians were reported for the ten-launch design. The per-binary relative change was

$$\Delta_{\mathrm{rel}}(M) = 100 \times \frac{M_{\mathrm{CM}} - M_{\mathrm{baseline}}}{M_{\mathrm{baseline}}}, \tag{3}$$

where M_{baseline} and M_{CM} denote the disabled- and enabled-condition medians, respectively. The expression requires a nonzero baseline median. Group summaries are medians of the per-binary changes, with each binary contributing one value per applicable metric and countermeasure.

Wall-clock time describes applicable short-lived executions. Cumulative CPU time and peak working-set memory describe persistent executions during the 10-s window. At least one applicable metric was available for 144 of the 155 combinations.

The 11 combinations without an applicable metric involved `ProcessEnum`. Ten involved short-lived benign binaries that completed before the required measurements were collected. One involved a persistent malicious binary with incomplete resource collection. The malicious persistent resource medians for this module therefore use nine samples.

Metric coverage and group-summary availability refer to different analysis levels. Coverage counts binary-countermeasure combinations with at least one applicable metric. N/A identifies a particular metric-specific group summary that was not retained. Consequently, overall coverage does not provide the denominator of every table cell, and N/A is not interpreted as zero change or countermeasure failure. Complete per-cell inclusion counts remain limited by the retained records.

Supplementary timing, interface, combined-activation, and behavioral tests retain their individually stated conditions, repetition counts, and observation windows. Their observations are not pooled with the historical resource campaign or counted as additional repetitions of its measurements.

Tabela 11 Median per-binary relative changes in operational metrics and availability of at least one metric per binary-countermeasure combination. Changes are percentages relative to the disabled condition.

	Time (%) Short-lived		CPU time (%) Persistent		Peak WS (%) Persistent		Any metric available
Countermeasure	Ben. (10)	Mal. (4)	Ben. (7)	Mal. (10)	Ben. (7)	Mal. (10)	Pairs
`SanitizePin EnvVars`	−3.5	−2.7	+0.2	0.0	0.0	+0.1	31/31
`InstMemcmp Mask`	−3.9	+0.9	+0.2	+0.3	−1.3	+0.3	31/31
`SkewMask`	−0.2	N/A	−38.5	−29.2	−87.2	−74.4	31/31
`AntiDebug`	−0.7	+6.8	−0.4	0.0	−0.2	0.0	31/31
`ProcessEnum`	N/A	N/A	−7.2	−6.7	−12.8	−7.5	20/31

Note: Ben.: benign; Mal.: malicious; WS: working set. CPU time is cumulative. Values are medians of per-binary relative changes in the ten-launch-per-condition design. N/A denotes an unavailable group summary.

6.3 Behavioral Interpretation of Operational Effects and Practical Benefits

Table 11 presents the operational measurements and the availability of at least one metric per binary-countermeasure combination. For persistent binaries, the median relative changes under `SkewMask` were −87.2% and −74.4% in peak working-set memory for benign and malicious programs, respectively. The corresponding changes in cumulative CPU time were −38.5% and −29.2%. These values describe resource differences during the 10-s observation window and do not establish performance improvements or equivalent work.

Parenthetical counts represent the reported profile sizes and do not necessarily provide valid denominators for individual cells. The last column counts combinations with at least one applicable metric out of the 31 evaluated per module (144/155 overall), without indicating the contributing count for each metric. The malicious persistent resource medians for `ProcessEnum` use nine samples. N/A does not indicate zero change or, by itself, countermeasure failure. Availability criteria are described in Section 6.2.

The available records do not determine whether the resource differences under SkewMask resulted from changes in progress, waiting periods, executed paths, or other application states. Remaining active during the observation window constitutes an operational observation without establishing equivalent behavior. Comparable task completion and longitudinal tracking of child processes after parent termination were not demonstrated.

To provide additional behavioral observations, supplementary diagnostic runs compared five benign x64 binaries under native execution and TMW0, with one run per condition and a 10-s cutoff. The TMW0 runs used `-q -mc 1`, with all countermeasures disabled, on the historical campaign build. Table 12 identifies the binaries and presents the reported exit codes, elapsed times, and observable agreement.

Tabela 12 Reported benign native/TMW0 observations: one run per condition and a 10-s cutoff.

Binary / hash prefix	Native	TMW0	Observable agreement
BEN_05	Exit 1	Exit 1	
0244aa82	2.7 s	8.5 s	Same MUI diagnostic on stderr.
BEN_06	Exit 3	Exit 3	
0436f442	0.5 s	7.0 s	Matching exit code; stdout empty.
BEN_10	Exit 0	Exit 0	
06418110	0.5 s	2.7 s	Same file-not-found message after normalization of the run-specific path.
BEN_12	Exit 0	Exit 0	
070a98b4	0.4 s	2.6 s	Same MAKECAB usage output.
BEN_16	Exit 0	Exit 0	
07c1f3b3	0.4 s	3.3 s	Matching exit code; stdout empty.

Note: Identifiers are accompanied by the reported SHA-256 prefixes. Times represent elapsed seconds in the diagnostic runs. These observations are separate from the ten-launch-per-condition resource campaign and are not used to estimate countermeasure overhead.

Exit codes agreed in all five pairs. Three pairs also produced corresponding diagnostic or usage output; the other two provide exit-code agreement only. Reproducing error or usage messages does not establish completion of productive work. Because the comparison involves native execution and TMW0, its results do not explain the resource differences between TMW0 and SkewMask or establish comparable progress in persistent programs.

6.4 Controlled Behavioral Recovery Demonstrator

The authors developed a benign x64 demonstrator, `TestRecoverWork.exe`, to examine, under controlled conditions, the relationship between a supported inspection, the selected branch, and an observable file operation. Its reported SHA-256 prefix is `5DC7AAC8`.

The program enumerates processes through `CreateToolhelp32Snapshot`, `Process32FirstW`, and `Process32NextW`, searching for `pin.exe`. When this name is visible, it selects the `EVASIVE` branch, skips the file operation, and exits with code 2. Otherwise, it selects the `WORK` branch, copies a fixed 281-byte input to `work.bin`, and exits with code 0.

The comparison used native execution, TMW0 (`-q`), and ProcessEnum enabled (`-dp -q`), with one recorded execution per condition and a 30-s cutoff. The instrumented conditions used the historical build identified in Section 7.4. Table 13 summarizes the reported observations.

Under native execution, the program selected the work branch and produced the file. Under TMW0, visibility of the monitored name led to selection of the evasive branch and absence of the file. With ProcessEnum enabled, the name was filtered,

Tabela 13 Controlled relationship between process-name inspection, selected branch, and file output: one execution per condition.

Condition	Detects `pin.exe`	Branch	SHA-256 prefix	Exit code
Native	No	`WORK`	`DAD7D83B`	0
TMW0	Yes	`EVASIVE`	Absent	2
ProcessEnum	No	`WORK`	`DAD7D83B`	0

Note: The SHA-256 column presents the reported hash prefix of `work.bin`, without independent recomputation of the full hash. "Absent" indicates that the file was not produced. The ProcessEnum execution reported 144 Tool Help calls and one filtered entry. Run-ID prefixes are native: `56c3df9c`; TMW0: `bd2aa90f`; and ProcessEnum: `09f1a9d8`.

the work branch was selected again, and the file operation was restored with the same reported hash prefix as in native execution.

This result provides controlled evidence linking indicator filtering to recovery of the operation defined in the demonstrator. Its construction by the authors limits generalization to independent programs. Confirmation of behavioral recovery in independently obtained malware or protected software remains an open validation step.

7 Discussion

7.1 Interpretation of Functional and Operational Results

Regarding RQ1, the controlled tests support suppression of the targeted indicators through the exercised interfaces. The four non-timing modules provide positive functional evidence within this scope. For `SkewMask`, the matched-workload comparison retains the same artificial workload in the evaluated conditions and supports the oracle outcome. However, obtaining a score within the threshold does not establish clock consistency or preservation of wait semantics.

Regarding RQ2, the historical campaign reports satisfaction of the adopted completion-or-observation criterion across all 155 binary-countermeasure combinations, with at least one applicable operational metric available for 144 combinations. The remaining 11 involve `ProcessEnum` and represent metric-availability limitations. These results characterize the recorded execution profiles without demonstrating equivalent useful work.

The median resource changes describe differences between the observed configurations. The larger reductions under `ProcessEnum` and `SkewMask` do not establish greater efficiency because comparable application progress was not demonstrated. Timing and suspension interventions may alter waiting periods, executed paths, and the state reached during measurement [6, 11, 20, 23]. However, the contribution of these factors to the observed reductions was not established by the available records.

The findings therefore distinguish targeted indicator suppression, operational compatibility, and resource differences, which do not necessarily represent the isolated

cost of a countermeasure. Their interpretation depends on the observation surface and the detection strategy used by the monitored program [9, 10].

7.2 Selective Activation and Practical Implications

Independent activation allows the analyst to select countermeasures associated with the investigated indicators and supports attribution of effects within controlled comparisons. The benign demonstrator developed by the authors illustrates this relationship by linking process-name filtering to a change in the executed branch and restoration of a file operation within the program constructed for the test.

Combined activation through `-da` was exercised in the five-detector smoke test using the historical build and in the focused comparison tests on the corrected build. These results provide evidence for the evaluated paths and combinations without characterizing interactions between modules across the full corpus.

The study did not experimentally compare TOMWare.M with Arancino [4] or other instrumentation and analysis solutions. Selective activation therefore constitutes an architectural and experimental property of the tool without establishing superior coverage, compatibility, or performance relative to external systems.

7.3 Limitations and Threats to Validity

Implementation coverage depends on the intercepted structures and routines. Comparison masking also depends on operand position and signature length. The benign-module control and the tests of four comparison routines do not independently validate every API variant, parameter combination, or error path. The corrected `_memicmp_l` test used a null locale argument. Its results apply to the separately identified build and do not retroactively validate this path in the historical campaign.

The timing tests exposed TickCount regressions in four of the ten compensated runs during overlapping waits. No alertable-wait return was observed before the test cutoff in any of the ten compensated runs. The causes remain unconfirmed, and the results do not establish correct accounting of overlapping delays or a globally coherent timeline. The intercepted QPC does not provide an independent external reference. Direct `RDTSC`/`RDTSCP` reads, external clocks, unsupported sources, absolute delays, and overhead outside intercepted relative waits remain outside coverage.

Construct validity is limited by the controlled oracles and the demonstrator developed by the authors. These tests describe relationships between checks and outcomes under the evaluated conditions without demonstrating that corpus binaries executed the same checks. The five benign diagnostic pairs provide limited agreement in exit codes and diagnostic or usage output. Comparable productive-task completion, longitudinal process-tree tracking, and behavioral recovery in independently obtained evasive software remain undemonstrated.

The corpus baseline includes Pin and TOMWare.M with the evaluated module disabled. Native and empty-Pintool controls were used in selected supplementary tests, but a complete baseline set was not collected for the corpus campaign or the timing series with 30 attempts per condition. Alternating condition order reduces ordering effects but is not equivalent to complete randomization.

Tabela 14 Identifiers, source references, and experimental use of the evaluated artifacts.

Artifact or reference	Identifier	Experimental use and provenance
Historical DLL (SHA-256)	`4BBC62707EE0B2EF922D4E8659A639` `92814EA0FC74F4682F1187562AE7F0`	Campaign of 31 binaries and 3,100 launches, timing tests, and controlled demonstrator.
Historical source (tag)	`jcvht-dll-4bbc6270-candidate`	Candidate reference; exact reconstruction of the historical DLL from this source has not been demonstrated.
Corrected source (commit)	`5cc893f6e2266f474e88` `945d6673e0c13e2cd683`	Source associated with the correction of the `_memicmp_l` wrapper.
Corrected DLL (SHA-256)	`D55EC952202F9E90C75FB5DDAEED1E` `85242B5D9CDA0337DA0C704690DBC1`	Focused masking and ordinary-call forwarding tests for four comparison routines.
Matched-workload timing CSV (SHA-256)	`76208218f71b37b8bf3411c0a74932` `2b96dd133d4db102eebbec25c65b98`	Three conditions with 30 attempts each; all 90 scores and collection flags were retained.

Note: Line breaks within identifiers are for presentation only; segments must be concatenated without spaces.

Statistical evidence differs between experiments. The timing series retains individual scores and Wilson confidence intervals for the proportion of attempts scoring at or below the threshold. Resource summaries remain descriptive medians, without reconstructed per-cell dispersion or confidence intervals. Corpus profile metadata and metric-inclusion counts require reconciliation. Build identities and historical provenance limitations are presented in Section 7.4.

External validity is limited by the 31-binary corpus, the x64 Windows configuration, the selected Pin version, and the virtualized infrastructure. Other programs and environments may expose different indicators and operational effects.

7.4 Reproducibility and Build Provenance

Table 14 lists the evaluated artifacts, their identifiers, and their experimental roles. This identification distinguishes historical-build results from those obtained after correcting the comparison wrapper.

The corrected build was evaluated separately. Its results neither replace nor retroactively validate the corpus, timing, or controlled-demonstrator measurements obtained with the historical DLL.

In addition to the 90 timing attempts, the complementary wait experiment retains 40 run records and their event logs. The documentation distinguishes experimental configurations, build identities, individual observations, and derived summaries.

The retained records do not yet permit complete reconstruction of the historical resource table or full reconciliation of corpus profile metadata. These constraints limit the reproducibility of the historical campaign.

8 Conclusion and Future Work

This article presented TOMWare.M, a countermeasure-oriented *Pintool* developed on Intel Pin for Windows. Its five independently selectable modules address PEB-based debugger indicators, process and module enumeration, Pin-related environment variables, exact memory-signature comparisons, and timing deviations.

The controlled tests support indicator suppression through the exercised interfaces. A separately identified build corrected an argument-count defect in the `_memicmp_l` wrapper and passed the focused comparison controls. The matched-workload timing experiment retained all 90 attempts. Under the same artificial workload, scores were at or below the threshold in 30/30 compensated attempts versus 11/30 without compensation. However, complementary tests exposed TickCount regressions in 4/10 compensated runs and alertable waits without an observed return before the test cutoff. Timing-indicator suppression therefore does not establish globally coherent clocks or preservation of wait semantics.

The historical campaign comprised 31 Windows PE binaries and 3,100 canonical launches. The records report satisfaction of the completion-or-observation criterion across all 155 binary-countermeasure combinations, with at least one applicable operational metric available for 144 combinations. These results do not demonstrate equivalent useful work or explain the large resource reductions observed under SkewMask.

A benign demonstrator developed by the authors linked process filtering to restoration of a controlled file operation. Five benign native/TMW0 pairs produced matching exit codes; three also produced corresponding diagnostic or usage output. Combined activation through `-da` completed the smoke test with all five detectors. These observations complement the controlled functional evidence, but behavioral recovery in independently obtained evasive software remains undemonstrated.

Immediate priorities are to investigate and correct the timing failures, revalidate the compensation model, and compare observable application milestones and process trees across conditions. Independent cases linking an instrumentation check, the selected branch, and recovered behavior are also needed. Validation should extend ordinary-call and error-path controls, complete the baseline conditions, reconcile corpus metadata, and verify the correspondence between the source code and the historical binary.

Future work will extend the evaluation of combined activation and interface coverage, including native enumeration mechanisms, additional debugger checks, and direct timing instructions such as `RDTSC` and `RDTSCP`.

Referências

[1] Luk, C.-K., Cohn, R., Muth, R., Patil, H., Klauser, A., Lowney, G., Wallace, S., Reddi, V.J., Hazelwood, K.: Pin: Building customized program analysis tools with dynamic instrumentation. In: PLDI '05, pp. 190–200. ACM, New York (2005). https://doi.org/10.1145/1065010.1065034

[2] Rodríguez, R.J., Rodríguez-Gastón, I., Alonso, J.: Towards the detection of isolation-aware malware. IEEE Latin America Transactions **14**(2), 1024–1036

(2016). https://doi.org/10.1109/TLA.2016.7437254

[3] Santos Filho, A., Rodríguez, R.J., Feitosa, E.L.: Evasion and countermeasures techniques to detect dynamic binary instrumentation frameworks. Digital Threats: Research and Practice **3**(2), 1–28 (2022). Article 11. https://doi.org/10.1145/3480463

[4] Polino, M., Continella, A., Mariani, S., D'Alessio, S., Fontana, L., Gritti, F., Zanero, S.: Measuring and defeating anti-instrumentation-equipped malware. In: DIMVA 2017. LNCS, vol. 10327, pp. 73–96. Springer, Cham (2017). https://doi.org/10.1007/978-3-319-60876-1_4

[5] Kim, M., Cho, H., Yi, J.H.: Large-scale analysis on anti-analysis techniques in real-world malware. IEEE Access **10**, 75802–75815 (2022). https://doi.org/10.1109/ACCESS.2022.3190978

[6] Park, S., Park, Y.: Detection techniques for DBI environment in Windows. Electronics **13**(5), 871 (2024). https://doi.org/10.3390/electronics13050871

[7] Lee, Y.B., Suk, J.H., Lee, D.H.: Bypassing anti-analysis of commercial protector methods using DBI tools. IEEE Access **9**, 7655–7673 (2021). https://doi.org/10.1109/ACCESS.2020.3048848

[8] Park, J., Jang, Y.-H., Hong, S., Park, Y.: Automatic detection and bypassing of anti-debugging techniques for Microsoft Windows environments. Advances in Electrical and Computer Engineering **19**(2), 23–28 (2019). https://doi.org/10.4316/AECE.2019.02003

[9] D'Elia, D.C., Coppa, E., Nicchi, S., Palmaro, F., Cavallaro, L.: SoK: Using dynamic binary instrumentation for security (and how you may get caught red handed). In: Asia CCS '19, pp. 15–27. ACM, New York (2019). https://doi.org/10.1145/3321705.3329819

[10] D'Elia, D.C., Invidia, L., Palmaro, F., Querzoni, L.: Evaluating dynamic binary instrumentation systems for conspicuous features and artifacts. Digital Threats: Research and Practice **3**(2), 1–13 (2022). Article 10. https://doi.org/10.1145/3478520

[11] Bruening, D., Zhao, Q., Amarasinghe, S.: Transparent dynamic instrumentation. In: VEE '12, pp. 133–144. ACM, New York (2012). https://doi.org/10.1145/2151024.2151043

[12] Padhy, S.T., Shanthi, P.B., Bijur, G., Hemalatha, S.: A threat-informed approach to malware evasion using DRM and TLS callbacks. IEEE Access **13**, 155195–155202 (2025). https://doi.org/10.1109/ACCESS.2025.3605020

[13] Falcón, F., Riva, N.: Dynamic Binary Instrumentation Frameworks: I Know You're There Spying on Me. REcon 2012. https://recon.cx/2012/schedule/events/216.en.html

[14] Kirsch, J., Zhechev, Z., Bierbaumer, B., Kittel, T.: PwIN: Pwning Intel Pin. In: ESORICS 2018. LNCS, vol. 11098, pp. 363–382. Springer, Cham (2018). https://doi.org/10.1007/978-3-319-99073-6_18

[15] Palmaro, F., Franchina, L.: Beware of unknown areas to notify adversaries. In: Intelligent Computing. LNNS, vol. 285, pp. 1003–1019. Springer, Cham (2021). https://doi.org/10.1007/978-3-030-80129-8_66

[16] Crăciun, V., Mogage, A., Lucanu, D.: Full Transparency in DBI Frameworks. arXiv:2306.13529 (2023). https://doi.org/10.48550/arXiv.2306.13529

[17] Gaber, M., Ahmed, M., Janicke, H.: Defeating evasive malware with Peekaboo: Extracting authentic malware behavior with dynamic binary instrumentation. Journal of Information Security and Applications **95**, 104290 (2025). https://doi.org/10.1016/j.jisa.2025.104290

[18] Shi, H., Mirkovic, J.: Hiding debuggers from malware with Apate. In: SAC '17, pp. 1703–1710. ACM, New York (2017). https://doi.org/10.1145/3019612.3019791

[19] Zhang, F., Leach, K., Stavrou, A., Wang, H.: Towards transparent debugging. IEEE Transactions on Dependable and Secure Computing **15**(2), 321–335 (2018). https://doi.org/10.1109/TDSC.2016.2545671

[20] Norby, A., Rimal, B.P., Brizendine, B.: Measurement of anti-debugging techniques on the Windows and Linux operating systems for the Intel x86_64 architecture. IEEE Access **13**, 46568–46583 (2025). https://doi.org/10.1109/ACCESS.2025.3550009

[21] Zeng, J., Fu, Y., Lin, Z.: PEMU: A Pin highly compatible out-of-VM dynamic binary instrumentation framework. In: VEE '15, pp. 147–160. ACM, New York (2015). https://doi.org/10.1145/2731186.2731201

[22] Zhao, C., Beugin, Y., Noirot Ferrand, J.-C., Burke, Q., Li, G., McDaniel, P.: LibIHT: A hardware-based approach to efficient and evasion-resistant dynamic binary analysis. In: SURE 2025, pp. 89–101. ACM (2025). https://doi.org/10.1145/3733822.3764670

[23] Oyama, Y.: How does malware use RDTSC? In: DIMVA 2019. LNCS, vol. 11543, pp. 197–218. Springer, Cham (2019). https://doi.org/10.1007/978-3-030-22038-9_10